



# 南京大學

## 本科畢業設計

院 系 智能軟件工程學院

專 業 軟件工程

題 目 基於 k-kick 樹的最優時空權衡

哈希表的設計與實現

年 級 2022 學 號 221900387

學生姓名 肖揚

指導教師 劉明謀 職 稱 副教授

提交日期 2026 年 6 月 3 日





# 南京大学本科毕业论文（设计） 诚信承诺书

本人郑重承诺：所呈交的毕业论文（设计）（题目：基于 k-kick 树的最优时空权衡哈希表的设计与实现）是在指导教师的指导下严格按照学校和院系有关规定由本人独立完成的。本毕业论文（设计）中引用他人观点及参考资源的内容均已标注引用，如出现侵犯他人知识产权的行为，由本人承担相应法律责任。本人承诺不存在抄袭、伪造、篡改、代写、买卖毕业论文（设计）等违纪行为。

作者签名：肖 扬

学号：221900387

日期：2026 年 5 月 29 日



# 南京大学本科生毕业论文（设计、作品）中文摘要

**题目：**基于 k-kick 树的最优时空权衡哈希表的设计与实现

**院系：**智能软件工程学院

**专业：**软件工程

**本科生姓名：**肖扬

**指导教师（姓名、职称）：**刘明谋 副教授

**摘要：**

Bender 等人在 2021 年提出了一种名为 k-kick 树的哈希方案<sup>[1]</sup>，其理论分析表明，若将哈希表的更新时间从  $O(1)$  放宽至  $O(k)$ ，可将每键空间浪费从  $\Omega(\log \log n)$  压缩至  $O(\log^{(k)} n)$  比特。本文依据此方案做了工程实现和实验验证。实现方面，以 C++ 构建了一个分层的弹性哈希存储结构，约 1065 行纯头文件模板代码。包含基于优先级的踢出机制、用 MiniArray 位压缩数组实现的紧凑二叉字典树路由、以及一个 8-bit Lookup Table 用于跳过字典树前 8 层加速查询。槽元数据和空闲槽位图分别压至每槽 4 比特和 1 比特。实验方面，在良好哈希条件下取  $n \in [5 \times 10^4, 10^6]$  五个规模，以 `std::unordered_map` 为对照，每组 5 次独立试验，测量插入、查询、删除和内存。 $n \leq 2 \times 10^5$  时，插入约 0.66–1.09 us/op，查询约 0.24–0.43 us/op，内存约 44–46 B/key； $n = 10^6$  时因合并摊还，插入升至约 7.47 us/op，查询约 1.59 us/op，内存约 102 B/key。高碰撞条件下，`std::unordered_map` 所有键被打入同一桶， $n = 10^5$  时查询退化至 132.5 us/op，而本方案保持在 0.39 us/op，差距约 342 倍。此外，将方案以适配器方式接入 LevelDB 内存表层，提供 CMake 编译开关和运行时开关。适配层单次 Hash 转换约 5.17 ns， $n = 2 \times 10^5$  时查询约 0.35 us/op。需要说明的是，本实现未完整复刻论文中的 per-cubby 多路由器架构等细节，实验数据反映的是当前简化版本的表现。

**关键词：**哈希表；时空权衡；k-kick 树；位压缩路由；LevelDB



# 南京大学本科生毕业论文（设计、作品）英文摘要

THESIS: Design and Implementation of Optimal Time/Space Tradeoff Hash Table

Based on  $k$ -kick Trees

DEPARTMENT: School of Intelligent Software and Engineering

SPECIALIZATION: Software and Engineering

UNDERGRADUATE: Yang Xiao

MENTOR: Associate Professor Mingmou Liu

ABSTRACT:

Bender et al. (2021) proposed a  $k$ -kick tree hashing scheme<sup>[1]</sup> whose theoretical analysis shows that relaxing update time from  $O(1)$  to  $O(k)$  reduces per-element space waste from  $\Omega(\log \log n)$  to  $O(\log^{(k)} n)$  bits. This thesis implements the scheme in C++ and evaluates it experimentally. The implementation comprises four 纯头文件模板 components (1065 lines total): a priority-based kicking mechanism, a compact binary trie local query router built on `MiniArray` bit-packing (slot metadata at 4+1 bits per slot), and an 8-bit lookup table to skip the first 8 trie levels during query. Experiments with  $n \in [5 \times 10^4, 10^6]$  key-value pairs (5 trials each) compare against `std::unordered_map` on insertion, query, deletion and memory. Under good hashing with  $n \leq 2 \times 10^5$ , insertion takes 0.66–1.09 us/op, query 0.24–0.43 us/op, and memory 44–46 B/key, at  $n = 10^6$ , insertion reaches 7.47 us/op, query 1.59 us/op, and memory 102 B/key. When all `std::unordered_map` keys are forced into a single bucket ( $n = 10^5$ ), query time degrades to 132.5 us/op while the scheme stays at 0.39 us/op, a 342x gap. The scheme is also integrated into LevelDB’s MemTable with a `-DUSE_KICK_HASH` compile-time switch and a `SetUseKickHash()` runtime toggle. Adapter overhead is 5.17 ns/call, query latency at  $n = 2 \times 10^5$  is 0.35 us/op. Note that this implementation does not fully replicate all details of the original paper (e.g., per-cubby multi-router architecture), so the experimental data reflects the current simplified version.

KEYWORDS: Hash table; Time-space tradeoff;  $k$ -kick tree; Elastic hashing; Bit-packed routing; LevelDB



# 目 录

|                          |   |
|--------------------------|---|
| 目 录                      | V |
| 第一章 引言                   | 1 |
| 1.1 研究背景和意义              | 1 |
| 1.2 研究现状                 | 2 |
| 1.2.1 链式寻址法              | 2 |
| 1.2.2 开放寻址法              | 2 |
| 1.2.3 布谷鸟哈希              | 2 |
| 1.2.4 基于 k-kick 树的时空最优方案 | 2 |
| 1.2.5 紧凑字典与空间高效结构        | 3 |
| 1.3 主要研究内容和目标            | 3 |
| 1.4 需求分析                 | 3 |
| 1.4.1 功能需求               | 3 |
| 1.4.2 非功能需求              | 4 |
| 1.4.3 用例分析               | 4 |
| 1.5 论文结构                 | 4 |
| 第二章 相关理论基础               | 5 |
| 2.1 哈希表基本概念              | 5 |
| 2.2 布谷鸟哈希                | 6 |
| 2.3 k-kick 树哈希方案         | 7 |
| 2.3.1 多层次 bin 划分         | 7 |
| 2.3.2 优先级踢出策略            | 7 |
| 2.3.3 理论保证               | 7 |
| 2.4 LevelDB 存储引擎概述       | 8 |

|                           |                                           |           |
|---------------------------|-------------------------------------------|-----------|
| 2.4.1                     | 存储架构 . . . . .                            | 9         |
| 2.4.2                     | 内存表的性能瓶颈 . . . . .                        | 9         |
| <b>第三章 系统设计 . . . . .</b> |                                           | <b>10</b> |
| 3.1                       | 总体架构 . . . . .                            | 10        |
| 3.1.1                     | 技术选型说明 . . . . .                          | 11        |
| 3.2                       | MiniArray: 位压缩数组 . . . . .                | 11        |
| 3.2.1                     | 设计动机 . . . . .                            | 11        |
| 3.2.2                     | 双模存储 . . . . .                            | 12        |
| 3.3                       | LocalQueryRouter: 紧凑二叉字典树局部查询路由 . . . . . | 12        |
| 3.3.1                     | 设计原理 (论文 §4.3 + Lemma 15) . . . . .       | 12        |
| 3.3.2                     | 紧凑节点编码 . . . . .                          | 12        |
| 3.3.3                     | Lookup Table 加速 . . . . .                 | 13        |
| 3.3.4                     | 两级路由中的使用 . . . . .                        | 14        |
| 3.4                       | Cubby: 单级弹性哈希表 . . . . .                  | 14        |
| 3.4.1                     | 槽元数据编码 . . . . .                          | 14        |
| 3.4.2                     | 踢出算法 . . . . .                            | 15        |
| 3.4.3                     | 容量尺度 . . . . .                            | 16        |
| 3.5                       | Facility: 全局分层管理器 . . . . .               | 17        |
| 3.5.1                     | 合并机制的设计考量 . . . . .                       | 17        |
| 3.5.2                     | 插入/查询/删除流程 . . . . .                      | 18        |
| <b>第四章 算法实现 . . . . .</b> |                                           | <b>19</b> |
| 4.1                       | 文件组织与依赖关系 . . . . .                       | 19        |
| 4.2                       | MiniArray 实现 . . . . .                    | 19        |
| 4.2.1                     | 位操作原语 . . . . .                           | 19        |
| 4.2.2                     | 模式切换 . . . . .                            | 20        |
| 4.3                       | LocalQueryRouter 实现 . . . . .             | 20        |
| 4.3.1                     | 紧凑节点编码 . . . . .                          | 20        |
| 4.3.2                     | 核心算法 . . . . .                            | 21        |
| 4.3.3                     | 显式双指针与查找表协同 . . . . .                     | 22        |

|            |                             |           |
|------------|-----------------------------|-----------|
| 4.4        | Cubby 实现 . . . . .          | 23        |
| 4.4.1      | bin 参数计算 . . . . .          | 23        |
| 4.4.2      | 深度分配 . . . . .              | 23        |
| 4.4.3      | 空闲槽查找 . . . . .             | 23        |
| 4.5        | Facility 实现 . . . . .       | 24        |
| 4.5.1      | 两级路由查询 . . . . .            | 24        |
| 4.5.2      | 合并与路由更新 . . . . .           | 24        |
| 4.6        | 复杂度分析 . . . . .             | 24        |
| <b>第五章</b> | <b>实验与分析 . . . . .</b>      | <b>26</b> |
| 5.1        | 实验设置 . . . . .              | 26        |
| 5.1.1      | 理论依据与规模选取 . . . . .         | 26        |
| 5.1.2      | 硬件与软件环境 . . . . .           | 26        |
| 5.2        | 良好哈希条件下的综合性能 . . . . .      | 27        |
| 5.2.1      | 插入性能 . . . . .              | 28        |
| 5.2.2      | 查询性能 . . . . .              | 28        |
| 5.2.3      | 删除性能 . . . . .              | 29        |
| 5.2.4      | 内存占用 . . . . .              | 30        |
| 5.3        | 高碰撞条件下的查询退化对比 . . . . .     | 31        |
| 5.3.1      | 实验设计与动机 . . . . .           | 31        |
| 5.3.2      | 实验结果 . . . . .              | 31        |
| 5.3.3      | 分析 . . . . .                | 32        |
| 5.4        | 数据汇总分析 . . . . .            | 33        |
| <b>第六章</b> | <b>LevelDB 集成 . . . . .</b> | <b>35</b> |
| 6.1        | 集成流程与适配器设计 . . . . .        | 35        |
| 6.2        | 双向开关设计 . . . . .            | 36        |
| 6.3        | 集成验证 . . . . .              | 37        |
| <b>第七章</b> | <b>总结与展望 . . . . .</b>      | <b>38</b> |
| 7.1        | 工作总结 . . . . .              | 38        |

|                     |    |
|---------------------|----|
| 7.2 理论与实践 . . . . . | 39 |
| 7.3 不足与展望 . . . . . | 39 |
| 参考文献 . . . . .      | 41 |
| 致 谢 . . . . .       | 45 |

# 第一章 引言

## 1.1 研究背景和意义

哈希表是数据库和搜索引擎中常见的数据结构，在理想情况下，插入、删除和查询都应达到  $O(1)$  时间。为了维持这个性能水平，实际系统中的哈希表需要预留多余的槽位空间来处理冲突。Bloom<sup>[2]</sup>在 1970 年分析了哈希编码在允许误差前提下的时空权衡关系，此后哈希表的空间效率一直是设计者要面对的问题。近年来，非易失性内存技术的发展<sup>[3]</sup>将存储层次推向更复杂的格局，对内存数据结构的空间效率和持久性提出了更高要求。

Google LevelDB<sup>[4]</sup> 和 Apache Cassandra 等日志结构合并树存储引擎已在工业界有大规模部署<sup>[5]</sup>。Luo 和 Carey 的综述指出，日志结构合并树的写放大主要来自多层归并，而内存表层的索引效率直接影响写入吞吐的上限<sup>[5]</sup>。已有研究尝试在跳跃链表上附加辅助索引来加快速度查询，但空间开销上仍有改进余地。

理论方面，早期工作<sup>[6-7]</sup>的结论是：如果更新必须在  $O(1)$  时间内完成且只用简单探测序列，键均空间冗余很难低于  $\Omega(\log \log n)$  比特。1984 年 Fredman、Komlós 和 Szemerédi 提出了 FKS 完美哈希方案<sup>[7]</sup>，在  $O(1)$  最坏查询下拿到线性总空间。但在此后的近四十年中， $\Omega(\log \log n)$  这个界限一直没有被突破<sup>[1]</sup>。

2021 年，Bender 等人<sup>[1]</sup>提出了一种命名为  $k$ -kick 树的哈希方案。该方案放宽了更新时间的约束从  $O(1)$  提升到  $O(k)$  把每键空间浪费从  $\Omega(\log \log n)$  压缩到  $O(\log^{(k)} n)$  比特。这里  $\log^{(k)} n$  表示对  $n$  迭代做  $k$  次对数： $\log^{(k)} n = \log(\log^{(k-1)} n)$ ，其中  $\log^{(0)} n = n$ 。当  $k = \log^* n$ （对  $n$  连续取对数直至结果  $\leq 1$  的次数）时，浪费可以降到  $O(1)$  比特。

本毕业设计的工作是：以 C++ 实现 Bender 等人的  $k$ -kick 树哈希算法<sup>[1]</sup>，做实验测量它在不同条件（包括极端碰撞）下的表现，并将这套实现以适配器方式接入 Google LevelDB，测试其在实际存储引擎中的可用性与性能。

## 1.2 研究现状

当前内存键值存储系统中的哈希索引可大致分成四条路线：链式寻址、开放寻址、布谷鸟哈希及其变体、以及基于紧凑编码的方案。它们在空间、查询延迟和写入吞吐上各自做了不同的取舍。

### 1.2.1 链式寻址法

链式寻址为每个槽维护一条链表，同一槽的键值对串在链表里。实现简单，扩容方便，但每个节点要多存一个后继指针，且链表在堆上不连续，冲突链一长容易频繁缓存未命中，以至于查询性能下降。

### 1.2.2 开放寻址法

开放寻址在连续槽数组上用探测序列解决冲突。一般通过线性探测做法把元素全存在一个数组里，缓存局部性好。不过负载因子  $\alpha$  靠近 1 时探针次数急剧增加，通常得把  $\alpha$  压在 0.5–0.7 之间，以保证探针次数维持在常数级别。

### 1.2.3 布谷鸟哈希

布谷鸟哈希<sup>[8]</sup>给每个元素算多个候选位置，通过踢出和重定位保证  $O(1)$  查询。Kirsch 等人<sup>[9]</sup>加了预留缓存槽，Arbitman 等人<sup>[10]</sup>用队列化踢出来保证确定性最坏情况。Hopscotch Hashing<sup>[11]</sup>把开放寻址的缓存友好和布谷鸟的踢出机制合在一起，在保持高负载因子的同时做到了较好的并发性能。

### 1.2.4 基于 k-kick 树的时空最优方案

前面的方案都没能打破  $\Omega(\log \log n)$  每键空间界限。2021 年 Bender 等人的  $k$ -kick 树方案<sup>[1]</sup>首次从理论上突破了它。思路是把键-槽分配变成多层次“球进桶”：键有随机最大深度，深者优先，最多踢  $k$  次。对任意  $k \geq 1$ ，更新时间  $O(k)$ ，空间浪费  $O(\log^{(k)} n)$  比特，构成一条可调的时空权衡曲线。论文还证明了这条曲线对所有“增强开放寻址”框架是紧的<sup>[1]</sup>。

## 1.2.5 紧凑字典与空间高效结构

Raman 和 Rao<sup>[12]</sup> 的紧凑动态字典用递归位级编码，在  $O(1)$  查询下把空间压到信息论极限的常数倍。Belazzougui 等人的 Hash Displace Compress 方案<sup>[13]</sup> 通过哈希-置换-压缩三道工序在静态场景下达成了近最优的空间效率。Liu 等人<sup>[14]</sup> 把紧凑编码从字典扩展到过滤器。Bender 等人<sup>[1]</sup> 的贡献在于第一次给出了从  $k = 1$  到  $k = \log^* n$  可连续调节的时空权衡曲线。

## 1.3 主要研究内容和目标

本工作围绕以下四件事展开：

1. 核心数据结构实现。以 C++23 实现四个纯头文件模板组件：MiniArray 位压缩数组、LocalQueryRouter 紧凑二叉字典树、Cubby 弹性哈希桶、Facility 全局管理器。尽量贴近论文<sup>[1]</sup>的设计意图。
2. LevelDB 内存表适配。写一个 KickHash64 函数做分片到 64-bit hash 的类型映射，再封装成 KickHashTable。改内存表的 Add/Get 路径，加上 CMake 编译开关和运行时 SetUseKickHash() 接口。
3. 正确性自测。用边界测试验证踢出链长度、字典树路由的指纹碰撞率、空间用法随数据规模的变化趋势等几个维度，测试是否明显偏离理论。
4. 性能实验。以 `std::unordered_map` 为对照，在良好哈希 ( $n \in [5 \times 10^4, 10^6]$ ) 和极端碰撞 ( $n \in [10^3, 10^5]$ ) 两个条件下测插入、查询、删除和内存。每组 5 次取均值。集成部分单测适配层开销。

## 1.4 需求分析

### 1.4.1 功能需求

FR1: 插入键值对。以 32-bit 整数为基础测试类型，用优先级踢出策略处理冲突。

FR2: 查询键值对。通过两级字典树路由定位目标 Cubby 和槽位，如果成功命中则返回结果值，否则返回未找到目标。

FR3: 删除键值对。清理两级路由映射，释放对应槽位。

- FR4: 合并。层级活跃 Cubby 在超出原论文<sup>[1]</sup>提到的阈值时自动触发同层合并，数据向上迁移一个层级 (tier)，同步更新路由。
- FR5: LevelDB 集成。提供用于决定采用数据库原生哈希或新型哈希方案的 `-DUSE_KICK_HASH` 编译开关和 `SetUseKickHash()` 运行时开关。

## 1.4.2 非功能需求

- NFR1: 在良好哈希且  $n \leq 2 \times 10^5$  时，单次插入  $\leq 2$  us/op。
- NFR2: 良好哈希且  $n \leq 2 \times 10^5$  时，单次查询  $\leq 0.5$  us/op。
- NFR3: 元数据开销控制在字节级别，槽元数据  $\leq 5$  bit/slot，字典树节点  $\leq 8$  B。
- NFR4: 碰撞条件下查询不出现数量级退化。
- NFR5: 纯头文件模板 C++23，约 1065 行，组件松耦合。

## 1.4.3 用例分析

系统的主要参与者为应用程序开发者（通过 API 调用）和 LevelDB 存储引擎（通过适配器接口）。表1-1列出了主要用例。

| 用例编号  | 用例名称         | 简要描述                                                    |
|-------|--------------|---------------------------------------------------------|
| UC-01 | 插入新键值对       | 用户调用 <code>Facility::insert()</code> ，系统通过两级路由和踢出链完成插入  |
| UC-02 | 查询键值对        | 用户调用 <code>Facility::lookup()</code> ，系统通过字典树路由定位槽位并返回值 |
| UC-03 | 删除键值对        | 用户调用 <code>Facility::remove()</code> ，系统清理两级路由并释放槽位     |
| UC-04 | 自动合并         | 当层级活跃 Cubby 超限，Facility 自动触发合并与路由更新                     |
| UC-05 | LevelDB 集成查询 | LevelDB 内存表通过 <code>KickHashTable</code> 适配器查询键值对       |
| UC-06 | 编译/运行时开关     | 开发者通过 CMake 选项或运行时接口启用/禁用 <code>KickHash</code> 加速      |

表 1-1 系统用例描述

## 1.5 论文结构

后面各章安排如下：第二章讲哈希表基本概念和 k-kick 树的理论基础；第三章是系统设计，从 MiniArray 往上一层层介绍四个组件；第四章是具体的实现细节和复杂度；第五章放实验数据和分析；第六章说 LevelDB 的集成；第七章做总结，用以罗列当前实现的不足以及后续改进方向。

## 第二章 相关理论基础

### 2.1 哈希表基本概念

哈希表通过哈希函数  $h : U \rightarrow [m]$  将键映射到有限个槽位置，支持插入、查询和删除三种操作，理想情况下均为  $O(1)$  期望时间。当  $h(k_1) = h(k_2)$  且  $k_1 \neq k_2$  时发生冲突，冲突处理策略直接影响哈希表在特定负载下的综合表现。

冲突处理策略大体分为链地址法和开地址法两类。链地址法将冲突键串入链表，允许负载因子  $\alpha$  超过 1，但每次查询伴随指针追逐和缓存未命中。开地址法则将所有键存入表本身，冲突时沿探测序列寻找替代位置。以线性探测为例，在良好哈希条件下： $\alpha \leq 0.7$  时平均探测次数低于 2； $\alpha = 0.9$  时升至约 5； $\alpha \rightarrow 1$  时探测次数发散<sup>[15]</sup>。链地址法下， $\alpha = 1.0$  时平均链表长度仅为 1， $\alpha = 2.0$  时升至 2，查询时间随  $\alpha$  线性增长而非超线性发散。但在  $\alpha < 0.8$  的典型区间内，开地址法的连续内存布局和缓存局部性使其实际速度优于链地址法。 $\alpha = 0.5$  时每个槽有一半概率为空，线性探测期望步数约 1.5； $\alpha = 0.75$  时期望步数约 2.5，空间利用率提高 50% 而时间开销仅增加约 67%，这是一个常见的操作点。 $\alpha$  超过 0.9 后，每增加 1% 负载，探测步数增长远超线性，此区间的空间节省通常不足以补偿时间代价。这一分岔直接影响哈希表设计中的时空取舍：开地址法依赖于预留更多空槽来减少探测步数，链地址法则是通过链表指针来换取更高的容许负载。 $k$ -kick 树方案<sup>[1]</sup>属于开地址法的扩展，其空间节省恰恰来源于在约定探测步数，即允许的  $k$  步的踢出链（ $k$  层树）内完成插入。

一个工程可用的哈希函数须满足确定性、均匀性和高效性三个准则。通用哈希族<sup>[16-17]</sup>为均匀性提供了形式化定义：若对任意  $x \neq y$ ， $\Pr[h(x) = h(y)] \leq 1/m$ ，则  $\mathcal{H}$  为通用哈希族。本系统采用 `splitmix64`<sup>[18]</sup>作为最终化函数，其 64-bit 混合步骤具有优异的雪崩效应<sup>[19]</sup>，通过注入不同种子可生成多个统计独立的哈希函数<sup>[13,16]</sup>，能够比较有效地保证哈希结果地均匀分布。

负载因子  $\alpha = n/m$  衡量哈希表的拥挤程度，插入和查询的期望时间复杂度随

$\alpha$  单调递增。空间效率用每键浪费比特数  $r$  度量：存储  $n$  个元素的信息论极限为  $B$  比特，实际占用  $B + rn$  比特，则浪费  $r$  比特/键。Bender 等人<sup>[1]</sup>证明了  $r$  可从  $\Omega(\log \log n)$  压缩至  $O(\log^{(k)} n)$ 。

## 2.2 布谷鸟哈希

布谷鸟哈希<sup>[8]</sup>是一种开地址哈希方案，做法是每个键有  $d$  个候选位置（通常  $d = 2$ ），由  $d$  个独立哈希函数分别确定。当插入新键时：

1. 依次检查  $d$  个候选位置，找到第一个空位即插入
2. 若所有候选位置均被占用，随机选择一个候选位置上的现有键将其踢出这个位置，由新键占据该位置
3. 被踢出的键尝试在自己的其他候选位置中寻找空位
4. 重复上述过程，直到所有键都找到位置或达到预设的最大踢出次数

布谷鸟哈希的优势在于查询严格为  $O(1)$ （只需检查  $d$  个位置），但其负载因子通常受限（ $d = 2$  时约 50%）。Arbitman、Naor 和 Segev<sup>[10]</sup> 进一步提出了利用队列技术去摊还布谷鸟哈希方案，通过队列化踢出操作缓冲区实现了确定性最坏情况性能保证，利用全局预取技术<sup>[16]</sup>控制踢出链长度，摆脱了经典方案对高概率分析的依赖。国内方面，邓莹莹<sup>[20]</sup>通过布隆过滤器和位图预判踢出必要性，在奇偶哈希表中降低了不必要的内存访问，提升了负载因子。金琪等人<sup>[21]</sup>提出了基于智能放置策略的 LBCHT 和 LPCCHT 两种变体，分别通过负载感知和空间局部性保持优化了 Cuckoo 哈希的写入长尾效应和查询效率。

布谷鸟哈希的踢出链存在一个隐患：键  $A$  踢出  $B$ 、 $B$  踢出  $C$ 、 $C$  踢出  $A$ ，三者的候选位置恰好形成闭合环，插入无法收束。这种情形在  $d = 2$  且负载接近 50% 时开始显著出现，原因在于每个键的两个候选位置构成了一个无向图，当图中出现环时踢出链可能沿环无限循环。对此有三种应对方式：设置最大踢出次数阈值，超限即触发全局重建，用新哈希函数重新分配所有键，或引入 stash，即一个容量较小的后备数组，遭遇循环的键直接存入 stash 不触发重建，以及增加哈希函数数量  $d$ ： $d = 3$  时负载上限约 91%， $d = 4$  时约 97%<sup>[22]</sup>，但每次查询需检查  $d$  个位置，常数开销相应增加。工程实践中， $d = 2$  配合约 50% 负载因子（查询优先）与  $d = 4$  配合约 95% 负载因子（空间优先）是两种常见配置。

## 2.3 k-kick 树哈希方案

Bender 等人<sup>[1]</sup>提出的  $k$ -kick 树方案是对布谷鸟思路的推广。其主要做法是：哈希表组织为多层级 bin 结构，优先级踢出策略保证每次插入至多  $k$  次移动。

### 2.3.1 多层级 bin 划分

将哈希表的  $m$  个槽组织为  $k + 1$  个层级（depth  $0, 1, \dots, k$ ）的 bin 层次结构。Depth-0 的 bin 包含全部  $m$  个槽，depth- $i$  ( $i > 0$ ) 的 bin 大小为  $s_i = \Theta((\log^{(i)} m)^6)$ 。随着深度增加，bin 大小呈指数级递减。这种层次结构被称为  $k$ -kick 树。

### 2.3.2 优先级踢出策略

每个键  $x$  被随机分配一个最大深度  $s(x)$ ，满足<sup>[1]</sup>  $\Pr[s(x) < i] = 1/(\log^{(i)} n)^2$ 。插入时按以下规则操作：

1. 新键  $x$  试图在尽可能深的 bin 中寻找空位
2. 若目标 bin 没有空位且处于没有饱和的状态（即存在深度较低的键）则踢出该低深度键，新键占据其位置
3. 被踢键保持原深度不变，在其自身深度 bin 中寻找新空位
4. 被踢键若触发新一轮踢出，则形成踢出链，但总踢出次数不超过  $k$ 。

关键保证：高深度的键优先踢出低深度的键，被踢键保持原深度。每次插入至多触发  $k$  次踢出，因此切换开销为  $O(k)$ 。

### 2.3.3 理论保证

论文<sup>[1]</sup>证明，对于任意  $k \in [\log^* n]$ ，上述方案可实现：

- 插入/删除时间： $O(k)$ （高概率）。
- 查询时间： $O(1)$ （确定性）。
- 每键空间浪费： $O(\log^{(k)} n)$  比特（高概率）。

此结果<sup>[1]</sup>突破了  $\Omega(\log \log n)$  的空间界限：每增加  $O(1)$  更新时间，空间浪费以  $\log^{(k)} n$  的速率递减。而参数  $k$  控制整条时空权衡曲线的形状。取  $k = 1$  时，空

间浪费为  $O(\log n)$  比特/键，与经典开地址法一致，插入至多 1 次踢出。 $k = 2$  时空间浪费压至  $O(\log \log n)$  比特/键，相当于 FKS 静态完美哈希<sup>[7]</sup>在动态条件下的表现，插入至多 2 次踢出。 $k = 3$  时进入  $O(\log \log \log n)$  区间， $k = 4$  时进一步缩至  $O(\log^{(4)} n)$ 。以  $n = 10^9$  为例： $\log n \approx 30$ ， $\log \log n \approx 5$ ， $\log \log \log n \approx 2.3$ ， $\log^{(4)} n \approx 1.2$ 。直观而言， $k$  每增加 1，空间浪费的函数阶数降低一级，那么插入时间就会增加常数步；对于实际可行的  $n$ （不超过  $2^{64}$ ）， $k \leq 4$  一般已经足以将空间浪费压缩至个位数的比特。

与标准布谷鸟哈希相比，本方案在踢出方向上存在一个结构性差异。布谷鸟哈希随机选取踢出对象，被踢键的替代位置同样随机，整个过程无方向性，依赖重试和全局重建来收束。 $k$ -kick 树方案则通过深度标记为踢出定义了优先级：每个键被分配最大深度  $s(x)$ ，高深度键优先踢出低深度键，被踢键维持原深度并在对应 bin 内重新定位。这种有向踢出策略保证踢出链始终指向更深层 bin，理论分析将其长度上界确定为  $k$ ，无需随机重试或全局重建。

这一结论对哈希表设计思路产生了深远影响。传统上，设计者将  $O(1)$  更新时间视为不可动摇的硬约束，在此前提下努力逼近  $\Omega(\log \log n)$  空间界限<sup>[6-7]</sup>。 $k$ -kick 树方案表明，一旦接受  $O(k)$  的更新时间（在现代硬件平台上  $k \leq 4$  时的实际延迟仍在微秒级），空间效率的提升远超常数因子优化的效果：不同  $k$  值对应不同的渐进函数类（ $\log n$ 、 $\log \log n$ 、 $\log \log \log n$  等），每个函数类与前一类的差距随  $n$  增长而发散。这种层次化的渐进改善是此前所有方案均未达成的。需要指出的是， $\Omega(\log \log n)$  界限是在“增强开放寻址”框架中证明的<sup>[1]</sup>；完美哈希方案（如 FKS<sup>[7]</sup>）虽然达到线性总空间，但依赖静态键集的预知和  $O(n)$  预处理时间，无法覆盖动态插入场景。

## 2.4 LevelDB 存储引擎概述

Google LevelDB<sup>[4]</sup>是一个由 Sanjay Ghemawat 和 Jeff Dean 开发的轻量级开源键值存储引擎，基于日志结构合并树架构<sup>[23]</sup>。其设计源自 Google Bigtable<sup>[24]</sup>中使用的底层存储架构，经模块化重构后作为独立库发布。

## 2.4.1 存储架构

LevelDB 的存储架构分为内存层和磁盘层两部分：

写入路径上，用户的插入以及删除操作先追加到磁盘预写日志（Write-Ahead Log，简称 WAL）保证持久性，随后键值对进入内存内存表。内存表基于跳跃链表<sup>[25]</sup>实现，查找  $O(\log N)$ ，追加插入  $O(1)$ 。当内存表写满（默认阈值 4 MB），它转为只读的、不可变的内存表，由后台压缩操作线程刷入磁盘生成序列化二进制表格文件。序列化二进制表格按层级组织：Level 0 的文件间 key 范围可重叠，Level 1 起逐层有序，每层容量约为上一层的 10 倍（Level 0 约 0.7 MB，Level 1 约 7 MB，依此类推）。压缩操作线程将低层序列化二进制表格与高层序列化二进制表格归并，删掉重复键和删除标记，输出新的有序序列化二进制表格。

LevelDB 的存储管理还包括版本控制（Version/VersionEdit）和元数据清单文件的元数据持久化。每次压缩操作完成后，版本控制记录本次归并涉及的文件变更（新增和删除的序列化二进制表格），以增量日志形式追加写入元数据清单文件。当前版本由 VersionSet 维护，内部以层级有序的序列化二进制表格集合描述系统状态，读查询遍历各层文件级元数据定位目标键可能存在的序列化二进制表格列表。此外，日志结构合并树的压缩操作策略（默认采用 Leveled 压缩操作）在高写入吞吐场景下可能产生写放大：写入 1 字节数据可能触发 10 倍以上的归并 I/O，这是日志结构合并树架构的核心设计取舍。

## 2.4.2 内存表的性能瓶颈

内存表位于 LevelDB 写入路径的最前端，其读写效率决定整个引擎的吞吐上限。默认跳跃链表实现提供  $O(\log N)$  查询复杂度，但在大规模数据（数百万键）和热点查询场景下，层间跳跃造成缓存行频繁失效，拖累点查询延迟。

Luo 和 Carey 的综述<sup>[5]</sup>指出，改进内存表的索引结构是降低日志结构合并树查询延迟的有效途径。本文将  $k$ -kick 树哈希方案集成至内存表层，作为跳跃链表的补充索引：维持原有插入性能不变，将点查询延迟从  $O(\log N)$  降至  $O(1)$ 。

## 第三章 系统设计

本章描述基于  $k$ -kick 树的哈希方案<sup>[1]</sup>的系统设计，自底向上逐一介绍四个组件：最底层为 MiniArray 位压缩数组原语，向上依次为 LocalQuery-Router 局部查询路由层、Cubby 单级弹性哈希表层，以及 Facility 全局分层管理器。四者通过模板参数和依赖注入松耦合连接。

### 3.1 总体架构

Facility 作为全局入口，维护一个 `cubbies_` 向量存放所有 Cubby，一个 `tail_idx_` 指向当前接收新数据的 tail Cubby，以及一个全局路由 `cubby_router_` 负责把键映射到对应的 Cubby 编号。

`cubbies_` 由多个层级的 Cubby 组成。tail 位于 tier-1，容量固定 1024，是唯一接收新插入的入口。tier-1 中填满的 Cubby 标记为非 tail，不再写入新数据。合并产物放在 tier-2 及更高层级，容量根据实际数据量动态指定。每个 Cubby 内部有独立的路由器 `route_` 把键映射到槽索引，还有 `slot_meta_` (MiniArray, 每槽 4 bit 记录占用状态和深度)、`free_bitmap_` (MiniArray, 每槽 1 bit 标识空闲)、`keys_[]` 和 `values_[]` 数组以及 bin 大小参数 `s_[]` 和 `k_`。

系统用两级路由组织键到槽的映射：

1. Cubby 级路由（由 `Facility::cubby_router_` 承担）：根据键的哈希值确定该键当前存储在哪个 Cubby 中。此路由为全局性映射，所有键共享同一个路由字典树实例。
2. Slot 级路由（由每个 Cubby 内部的 `Cubby::route_` 承担）：在通过第一级路由确定目标 Cubby 后，进一步将键映射到该 Cubby 内部的某个具体槽索引。每个 Cubby 拥有独立的路由实例，因此单个 Cubby 的路由操作不干扰其他 Cubby。

两级分离设计的好处是显而易见的：当某一个 Cubby 被合并或停用时，只需要更新 Cubby 级路由中对应的那部分键的映射信息，无需触碰其他活跃 Cubby 的内部 Slot 级路由。这种隔离性使得系统的合并操作成本与所合并的实际键数量呈线性关系，而非与全系统总键数相关。

### 3.1.1 技术选型说明

系统在语言选型、组织方式和算法方面采用以下技术方案：

- C++23 语言标准。系统使用 C++23 标准的 `constexpr`、`constexpr` 和 `std::bit_cast`，在编译期完成部分计算以降低运行时开销。MiniArray 的位宽参数通过模板在编译期确定。
- 纯头文件模板组织方式。四个组件均以纯头文件（.hpp）形式组织，无独立编译单元。使用者将头文件复制至项目目录并添加 `#include` 即可集成，同时编译器获得更大的内联优化空间。
- splitmix64 哈希函数。哈希函数使用 splitmix64<sup>[18]</sup>，而非 CityHash 或 xxHash 等重型方案。splitmix64 的 64-bit 雪崩混合步骤位扩散均匀，通过不同种子值可生成任意数量统计独立的哈希函数<sup>[16]</sup>，单次调用成本极低（约 5 ns），适合字典树查询路径上的高频调用。
- 与 `std::unordered_map` 的定位差异。

C++ 标准库的 `std::unordered_map`

基于链式寻址法，在良好哈希条件下性能出色，但其查询路径依赖单个哈希函数的均匀性，对恶意碰撞数据敏感。本方案的字典树路由将键的查询路径与其哈希比特串绑定，不同键的查询路径互不交叉，对抗性输入下退化上界为  $\log n$ ，优于 `std::unordered_map` 的  $O(n)$  退化。

## 3.2 MiniArray：位压缩数组

### 3.2.1 设计动机

在紧凑哈希表中，大量元数据（槽状态、路由条目）的位宽远小于标准整型（64 bits）。例如槽元数据仅需 4 比特（`occupied + depth`），路由条目中 cubby 索

引仅需 11 比特。若使用 `std::vector<uint64_t>` 存储，每个元素固定 64 比特，造成严重空间浪费。

MiniArray 将元素紧密平铺在连续位流中，按精确位宽存储<sup>[1]</sup>，其两种存储模式的特性见表 3-1。

| 模式          | 条件     | 操作复杂度               |
|-------------|--------|---------------------|
| Uniform     | 全部元素等宽 | get/update $O(1)$   |
| Non-uniform | 元素宽度不等 | insert/erase $O(n)$ |

表 3-1 MiniArray 双模存储

### 3.2.2 双模存储

- Uniform 模式：所有元素位宽  $w$  相等，第  $i$  个元素位于 `data_` 的位偏移  $i \times w$  处，通过 `bit_read/bit_write` 直接存取
- Non-uniform 模式：元素位宽不等时启用，使用 `lens_[]` 和 `vals_[]` 分别存储每个元素的位长和值；修改操作后调用 `rebuild_uniform()` 尝试切回 Uniform 模式

## 3.3 LocalQueryRouter：紧凑二叉字典树局部查询路由

### 3.3.1 设计原理（论文 §4.3 + Lemma 15）

论文<sup>[1]</sup>的局部查询路由器将每个键  $x$  哈希为 64-bit 全随机二元串  $r_x$ ，在集合  $S$  上构建最小唯一前缀二叉字典树  $T$ ：对每个  $r_x$ ，取最短的、不与任何其他  $r_y$  冲突的前缀作为该键的唯一标识。字典树的叶节点存储该键对应的 probe 值  $f(x)$ 。

论文 Lemma 15 证明：随机二元串的字典树期望规模为  $O(k)$  个节点（其中  $k = |S|$ ），且对任意常数  $c > 1$ ，存在  $\varepsilon > 0$  使得  $|T| \leq (\log n)/c$  以高概率成立。

### 3.3.2 紧凑节点编码

本次实现将字典树及 f-values 以紧凑位编码存储于 MiniArray 中：

- 节点编码：每个节点固定  $w = \text{entry\_bits\_}$  位宽，最高位为类型标志

(0= 内部节点, 1= 叶节点)。内部节点存储 `left_child + right_child` 两个显式子指针 (各 `child_bits_` 位), 叶节点存储 `f_value` (`fval_bits_` 位) + 键哈希指纹 (`kh_bits_` 位, 默认 44 位)。

- 查询 (`walk_down`): 沿键的最低有效位方向哈希位遍历字典树, 遇叶节点用指纹验证后返回 `f_value`。
- 插入 (`insert + do_split`): 沿路径 `walk` 遍历, 遇异键叶时分裂 (找两键哈希分叉位, 然后创建内部节点链, 最后在分叉处创建两新叶); 遇 `NO_CHILD` 空方向时链接新叶。
- 删除 (`remove`): `walk_down` 找到叶, 标记为空叶 (`hash=0, fval=0`)。

整体空间即  $|nodes\_| \times w$  bits, 其中  $|nodes\_| \approx 2|S|$ ,  $w \leq 62$  bits。紧凑编码实现了每节点 5-8 bytes 的空间占用。

### 3.3.3 Lookup Table 加速

论文<sup>[1]</sup>提出借鉴四俄罗斯人方法技术<sup>[26]</sup>构建预处理查找表 (lookup table), 将紧凑二叉字典树的查询复杂度从  $O(\log |S|)$  压至  $O(1)$ 。做法是: 对哈希值的前若干比特, 预计算每种位模式对应的字典树跳转目标节点, 查询时跳过这些比特, 直接从目标节点向下遍历。

本实现采用 8-bit 查找表: 预计算哈希低 8 位 (256 种前缀模式) 各自在字典树中可达的目标节点索引, 存入 `lut_[]`。查询时以键哈希低 8 位直接查表获得起跳节点, 跳过前 8 层, 从第 9 位起继续 `walk-down` 至叶节点。8-bit 查找表仅消耗  $256 \times 4B = 1KB$  空间, 将字典树查询深度缩减 8 层, 在本文实验规模 ( $n \leq 10^6$ , 字典树深度约 20 层) 削减约 40% 的遍历工作量。

以具体实例说明查找表的工作方式。设某键经 `splitmix64` 哈希后的低 8 位为 `0xA3` (二进制 10100011), `ensure_lut()` 在查找表构建阶段已从字典树根节点出发, 沿最低有效位方向按 `b0=1`、`b1=1`、`b2=0`、`b3=0`、`b4=0`、`b5=1`、`b6=0`、`b7=1` 的顺序下降 8 层, 将最终抵达的节点索引写入 `lut_[0xA3]`。查询时, `probe_index` 以键哈希的 `0xA3` 直接索引 `lut_[]` 数组, 一步跳至该预计算节点, 随后从该节点沿哈希第 9 位起继续逐位向字典树下层遍历。此过程以 1 KB 的查表空间换取字典树前 8 层的全部逐位穿越开销。

查找表在字典树节点总数 (`nodes_.size()`) 增长时失效：插入或分裂操作会增加节点，此时 `lut_valid_ = false`；键已存在时的值更新不改变拓扑，查找表保持有效。后续查询检测到失效后调用 `ensure_lut()` 重建，代价  $O(2^8 \times \text{depth}) = O(256 \times 8)$ 。查找表重建频率与字典树结构变更频率相当，远低于查询频次，摊还成本可忽略。

### 3.3.4 两级路由中的使用

- `Facility::cubby_router_`：  
全局二级路由，容量  $2^{21}$ ，映射键到 cubby 索引
- `Cubby::route_`：  
每 cubby 内一级路由，容量 `capacity`×2，映射键到槽索引

两者均使用同一 `LocalQueryRouter` 模板，无 `oracle` 时通过 `key_hash_val` 指纹验证键身份。

## 3.4 Cubby：单级弹性哈希表

Cubby 是 *k-kick* 树方案的物理载体<sup>[1]</sup>，也是系统中最复杂的单体组件。每个 Cubby 实例有独立的容量参数 *m* 可通过构造函数中的 `desired_capacity` 显式指定，也可由 `compute_capacity` 静态方法根据层级和全局数据规模自动推算并在此容量上构建 *k* + 1 个深度层级的 *kick-bin* 层次网络。

Cubby 的容量不是固定值。构造时允许传入动态指定的期望容量，这在合并操作频繁的分层管理架构中是必要的：两个层级处于第 *j* 层级的 Cubby 合并为一个层级处于 *j* + 1 层的 Cubby 时，新 Cubby 的容量综合考量论文理论公式的参考值与合并后实际数据量的两倍的经验界限，在避免空间透支的同时维持不低于 50% 的槽位利用率。

### 3.4.1 槽元数据编码

每槽使用 4 比特位压缩存储：

- bit 0: occupied (1 表示已占用)

- bit 1–3: depth (0–7, 指代该槽所在 kick 深度)

4 比特的取值有明确的信息论依据。depth 的取值范围为 0–7 ( $k_{\max} = 4$  时踢出深度最多  $k = 4$  层, 含 depth-0 的根 bin 共 5 层; 预留余量至 7 以兼容更大的  $k$ ), 需  $\lceil \log_2 8 \rceil = 3$  比特。occupied 仅为布尔标志, 需 1 比特, 合计 4 比特。若以常见的 `uint8_t` 逐槽存储元数据, 则每个 Cubby 中  $m$  个槽的元数据总开销为  $m$  字节; `MiniArray` 将其压缩至  $m/2$  字节, 节省恰好一半。对于典型 Cubby 容量  $m = 1024$ , 节省 512 字节;  $m = 65536$  时节省 32 KB, 其累积效应在多层 Cubby 并存时进一步放大。另设 1 比特 `free_bitmap_` 标识空闲槽 (1= 空闲), 配合 `first_free_` 缓存指针实现近似  $O(1)$  的空闲槽查找。

### 3.4.2 踢出算法

Cubby 的 `try_place` 实现了论文 §3.1<sup>[1]</sup>的优先级踢出:

---

**Algorithm 1** try\_place(key, max\_depth)

---

```
1: for  $d = \text{max\_depth}$  down to 0 do
2:    $\text{bin} \leftarrow \text{bin\_for\_key}(\text{key}, d)$ 
3:   Phase 1: 扫描  $\text{bin}$ , 若找到空槽则返回
4:   if  $d > 0$  then
5:     Phase 2: 扫描  $\text{bin}$  中深度  $< d$  的占槽
6:     if 找到 then
7:       暂存  $\text{victim\_key}, \text{victim\_val}$ , 清空该槽
8:       递归 try_place(victim, victim_depth, kick_idx)
9:       if 成功 then
10:        victim 移入新槽, 新键占据该槽
11:        return
12:       else
13:        恢复原槽
14:       end if
15:     end if
16:   end if
17: end for
18: return FAIL
```

---

被踢键保持原深度, 在其自身深度  $\text{bin}$  中寻找新空位。算法中的 `reserved` 参数防止被踢键在递归扫描时“发现”刚被清空的原槽。

### 3.4.3 容量尺度

基于论文公式  $C_j = K/(\log^{(j)} n)^2$  ( $K \approx 409600$ )<sup>[1]</sup> 计算各层容量。此外, 构造时支持传入 `desired_capacity` 以动态指定容量 (用于合并场景)。合并时的容量策略为  $\max(\text{论文公式}, \text{merged\_size} \times 2)$ , 取 2 的幂, 确保约 50% 的空间利用率。各层级对应的基准容量和场景见表3-2。

| Tier | 基准容量          | 场景              |
|------|---------------|-----------------|
| 1    | 1,024         | Tail, 新数据写入     |
| 2    | 约 4,096 (动态)  | 两个 tier-1 合并为一个 |
| 3    | 约 16,384 (动态) | 两个 tier-2 合并为一个 |
| 4    | 约 65,536 (动态) | 两个 tier-3 合并为一个 |

表 3-2 Cubby 容量尺度 (动态容量)

## 3.5 Facility: 全局分层管理器

Facility 组件负责全局状态协调、合并调度和统一查询接口<sup>[1]</sup>。内部维护着当前活跃的写入目标 (tail 指针)、各层级的 Cubby 计数、以及全局 Cubby 级路由表 `cubby_router_`。

### 3.5.1 合并机制的设计考量

当某一层级的活跃 Cubby 数量超过预设上限 (由 `max_cubbies` 函数根据层级编号返回的固定阈值) 时, Facility 触发该层级的合并流程。两个被选中的同层 Cubby 中的所有元素被全部提取, 依次插入一个新分配的、高一层级的 Cubby。每个被迁移的键都需在全局路由表中更新其 Cubby 索引映射。这是合并操作的主要时间开销。

为避免 `std::vector::erase` 引起的索引整体偏移 (会使已存储键的 Cubby 编号大规模失效, 触发全量路由重建), 本实现采用 `deactivate()` 停用而非物理删除。被停用的 Cubby 清空内部存储、将 `active_` 置为 `false`, 但其在 `cubbies_` 向量中的索引保留不变。后续分配新 Cubby 槽位时, `alloc_cubby_slot` 优先回收这些已停用的空槽, 实现索引循环利用。路由表中指向其他活跃 Cubby 的映射条目在整个合并过程中始终保持有效, 无需额外的批量修复操作。各层级的合并触发阈值见表3-3。

| Tier | max_cubbies | 触发条件            |
|------|-------------|-----------------|
| 1    | 200         | 约 20 万元素触发合并    |
| 2    | 50          | 50 个 tier-2 后合并 |
| 3    | 20          | 20 个 tier-3 后合并 |
| 4    | 5           | 5 个 tier-4 上限   |

表 3-3 合并阈值

### 3.5.2 插入/查询/删除流程

插入: `cubby_router_` 查重; 若 `tail` 满则执行 `promote` 和 `rebalance`; 然后调用 `Cubby::insert_new` (含 `kick` 链); 最后更新两级路由。

查询: `cubby_router_` 根据哈希值在字典树中路由定位 `cubby`, 然后再通过 `Cubby::route_` 定位到相应槽位, 最后调用 `lookup_at` 比对键值验证这个被查询的键值是否存在。

删除: `cubby_router_` 根据哈希值定位目标 `cubby`, 然后执行 `Cubby::remove_at` 移除键值对并释放槽位, 随后清理 `cubby_router_.remove` 更新路由表空闲标记。

## 第四章 算法实现

本章逐一介绍四个组件的主要算法流程和实现细节。代码使用 C++23 标准，全部纯头文件模板（纯头文件模板）组织，外部使用者直接包含 .hpp 文件即可获得完整定义，无需额外编译链接。总规模约 1065 行，外部依赖度较低。四个源文件的组织结构与职责划分见表4-1。

### 4.1 文件组织与依赖关系

| 文件                     | 行数  | 职责                               |
|------------------------|-----|----------------------------------|
| miniarray.hpp          | 261 | 位压缩数组，Uniform 和 Non-uniform 双模存储 |
| local_query_router.hpp | 350 | 紧凑二叉字典树局部查询路由 8-bit Lookup Table |
| cubby.hpp              | 419 | 单级弹性哈希表 free_bitmap 动态容量         |
| facility.hpp           | 233 | 全局分层管理两级路由惰性状态标记合并               |

表 4-1 源代码文件组织

四个头文件之间有单向依赖链：miniarray.hpp 为最底层，被其余三者依赖；local\_query\_router.hpp 依赖 miniarray.hpp；cubby.hpp 依赖前两者；facility.hpp 依赖全部三个底层组件。分层依赖保证编译的有序性和代码的可测试性。

### 4.2 MiniArray 实现

#### 4.2.1 位操作原语

`bit_read(bit_off, bit_len)`：从 `data_` 位偏移处读取指定位数。对于跨 64-bit 字的情况，通过两次读取和拼接实现。特判 `bit_len=64` 时使用 `~0ULL` 掩码，避免 `(1ULL << 64)` 的未定义行为。

`bit_write(bit_off, bit_len, value)`: 写入指定位数, 自动扩展 `data_` 容量。同样处理跨字写入和 64-bit 掩码。

Listing 4.1: `bit_read` 位读取实现

```
uint64_t bit_read(size_t bit_off, size_t bit_len) const {
    size_t word = bit_off / 64, shift = bit_off % 64;
    uint64_t lo = data_[word] >> shift;

    if (shift + bit_len <= 64)
        return lo & ((bit_len < 64) ? ((1ULL << bit_len) - 1)
            : ~0ULL);

    uint64_t hi = data_[word + 1] << (64 - shift);
    return (lo | hi) & ((1ULL << bit_len) - 1);
}
```

## 4.2.2 模式切换

`to_non_uniform()`: 将 Uniform 数据转为 Non-uniform 格式 (`lens_[]` + `vals_[]`)。 `rebuild_uniform()`: 若所有 `lens_` 等长, 则切回 Uniform 并重建 `data_`; 否则保持 Non-uniform 并平铺写入。

## 4.3 LocalQueryRouter 实现

### 4.3.1 紧凑节点编码

每个字典树节点在 `MiniArray` 中以固定的 `entry_bits_` 位宽 (不超过 62 位) 存储, 最高位为类型标志: 0 表示内部节点, 1 表示叶节点。内部节点将剩余位均分为左右两个子指针字段 (各 `child_bits_` 位), 分别指向左子树和右子树的根节点在 `MiniArray` 中的序号。叶节点将剩余位划分为功能值字段 (`fval_bits_` 位, 高位部分) 和键哈希指纹字段 (`kh_bits_` 位, 默认 44 位, 低位部分)。功能值存储该键在所属 `Cubby` 中的槽位索引, 键哈希指纹取值为 `splitmix64(hash(key)) & ((1<<44)-1)`, 用于查询时快速验证叶节点与查询键的匹配关系。子指针位宽由公式4-1 确定, 随路由器容量参数动态调整; 功能值位宽取决于使用场景 (`Cubby` 级路由中为槽位索引位宽,

Facility 级路由中为 Cubby 编号位宽)。类型位、子指针（或功能值及哈希指纹）拼接后得到完整节点编码，由 MiniArray 以 Uniform 模式紧凑存储。

$$\text{child\_bits\_} = \text{bits\_to\_represent}(\text{cap} \times 3 + \text{NODE\_PAD}) \quad (4-1)$$

### 4.3.2 核心算法

查询 (probe\_index): 对 key 计算哈希得到 64-bit 二元串，从根节点沿最低有效位遍历字典树。若查找表有效则跳过前 8 层直接跳转至目标节点。遇叶节点用指纹验证：匹配则返回 f\_value，否则返回 SIZE\_MAX。

插入 (insert): 沿 key 的最低有效位哈希位遍历字典树。五种情况：

1. 遇叶节点且指纹匹配：更新 f\_value
2. 遇叶节点且指纹为 0（空叶，首次使用）：覆盖为新键数据
3. 遇叶节点但指纹不匹配：执行 do\_split() 分裂。找分叉深度，创建内部节点链，在分叉处创建两新叶
4. 遇内部节点但方向为 NO\_CHILD：创建新叶并链接
5. 路径走到头：追加新叶节点

插入结束后若 nodes\_.size() 变化则失效查找表。

do\_split(leaf\_idx, depth, e\_hash, e\_fval, n\_hash, n\_fval):

1. 从 depth 起找两 hash 的首个分叉位 diverge
2. 从 depth 到 diverge-1 逐位创建内部节点链（每次 alloc\_node）
3. 若当前节点为原叶：转换为 internal；若为内部节点：更新对应方向子指针
4. 在 diverge 处创建 e\_leaf（已存键）和 n\_leaf（新键）
5. 设置最后一个内部节点的左右子节点为两叶

删除 (remove): 沿字典树 walk 找到匹配叶节点，写入空叶标记（即下述两个变量 fval=0, hash=0 的节点），n\_entries\_ 触发自减。

### 4.3.3 显式双指针与查找表协同

字典树节点存储采用显式 **left+right** 双指针：内部节点的数据字段由两个等宽的 `child_bits_` 位子指针（分别指向左、右子树的根节点序号）拼接而成。`do_split` 需要分配新节点时，`alloc_node()` 始终在 `MiniArray` 末尾追加，已有节点的子指针引用关系因此在整个分裂过程中保持稳定已有节点索引和父子关系不因新节点插入而偏移。这一稳定拓扑简化了分裂算法的实现，也使查找表的重建判定简化为仅检查节点总数是否变化。

查找表构建由 `ensure_lut()` 统一管理。该方法遍历哈希值低 8 位的全部 256 种前缀模式，对每种模式从根节点沿对应方向下降，直到抵达叶节点或遇到 `NO_CHILD` 空方向，将到达的节点索引记录于 `lut_[]` 的对应表项。若 `ensure_lut()` 在节点拓扑变更后首次调用，时间复杂度为  $O(256 \times L)$ ，其中  $L$  为 8 层字典树的平均遍历深度，在  $10^6$  规模下约为数千次位操作和数组读取，对整体性能影响微乎其微。

Listing 4.2: 查找表构建与查询

```
void \texttt{ensure_lut}() {
    if (lut_valid_) return;
    for (int prefix = 0; prefix < 256; ++prefix) {
        size_t cur = 0;
        for (int d = 0; d < 8; ++d) {
            auto e = \texttt{nodes_}.get(cur);
            if (is_leaf(e)) break;
            cur = hash_bit_at(prefix, d) ? right_child(e) :
                left_child(e);
        }
        lut_[prefix] = (cur != \texttt{NO_CHILD}) ? (int32_t)
            cur : -1;
    }
    lut_valid_ = true;
}
```

## 4.4 Cubby 实现

### 4.4.1 bin 参数计算

`compute_kick_params()`: 从 `capacity_` 出发迭代  $\lceil \log_2(x+1) \rceil$  得到  $k$  ( $\leq 4$ ), 对每层应用迭代对数压缩后平方取整得到 `s_[i]`。初始化 `free_count_[d]` 为各深度的总槽数。

### 4.4.2 深度分配

`max_depth_for(key)`: 对键做 `hash`, 使用概率公式  $\Pr[s(x) < i] = 1/(\log^{(i)} n)^2$  计算阈值, 从  $k$  到 0 逐层比较并返回第一个命中深度<sup>[1]</sup>。

### 4.4.3 空闲槽查找

`free_bitmap_(MiniArray, 1 bit/slot, 0 表示占用)` 配合 `first_free_` 缓存指针, 实现近似  $O(1)$  的空闲槽查找。`find_free_in_bin()` 从缓存的 `first_free_` 位置起扫描目标 `bin`, 插入后清 `bit`, 删除后置 `bit` 并更新缓存。

Listing 4.3: `find_free_in_bin` 空闲槽扫描

```
size_t find_free_in_bin(size_t bin_start, size_t bin_end,
    size_t reserved) {
    if (\texttt{first_free_} < bin_start) \texttt{first_free_}
        = bin_start;
    for (size_t i = \texttt{first_free_}; i < bin_end; ++i)
        if (\texttt{free_bitmap_}.get(i) && i != reserved) { \
            \texttt{first_free_} = i + 1; return i; }
    for (size_t i = bin_start; i < \texttt{first_free_}; ++i)
        if (\texttt{free_bitmap_}.get(i) && i != reserved) { \
            \texttt{first_free_} = i; return i; }
    return \texttt{SIZE_MAX};
}
```

## 4.5 Facility 实现

### 4.5.1 两级路由查询

lookup(key):

1. cubby\_router\_.probe\_index(key) 获取 cubby 索引 *ci*。
2. cubbies\_[ci].lookup\_slot(key) 获取槽索引 *slot*。
3. cubbies\_[ci].lookup\_at(slot, key) 获取返回值。

### 4.5.2 合并与路由更新

merge\_two\_of\_tier(tier): 选取两个候选 Cubby, 提取全部元素, 按实际数据量计算动态容量, 通过 alloc\_cubby\_slot 获得新 Cubby, 逐元素插入新 Cubby 并逐一更新 cubby\_router\_ 中键到 cubby 索引的映射, 最后 deactivate() 原 Cubby。使用 deactivate() 停用避免 vector::erase 带来的索引偏移。

## 4.6 复杂度分析

表4-2汇总了本系统中各主要操作在期望情况和摊还意义下的时间复杂度。括号内标注了造成该复杂度的主要原因。

| 操作                   | 期望                    | 摊还 (含合并)      | 主要原因                               |
|----------------------|-----------------------|---------------|------------------------------------|
| 查询                   | $O(\log  S )$ + 查找表加速 |               | 紧凑字典树 walk (查找表跳过前 8 层)            |
| 插入                   | $O(\log  S )$         | $O(m)$ / 每次合并 | $m$ 为合并涉及的实际元素数                    |
| 删除                   | $O(\log  S )$         |               | 两级紧凑字典树 walk 和标记空叶                 |
| 空闲槽查找                | 近似 $O(1)$             |               | first_free_ 缓存和 free_bitmap_       |
| 空间 ( $n \leq 200K$ ) | 约 45 B/键              |               | keys/values 向量大约 8 B and 字典树 5-8 B |

表 4-2 复杂度汇总

上表中插入操作的期望复杂度  $O(\log |S|)$  来自 Cubby 内字典树插入过程, 该过程沿哈希值的各比特位逐层下降以确定插入路径。 $O(m)$  的摊还成本则源于合并操作: 每当系统触发 Cubby 合并时, 需对已迁移至新位置的元素逐一重建字典树路由信息。惰性状态标记策略将合并成本限定为与被合并元素数成线性关系,

而非与系统内全部键的总数挂钩，这在大规模数据持续插入场景下有效抑制了合并成本的累积，使得整体性能衰减更为平缓。

查询操作的理论复杂度为  $O(\log |S|)$ ，即沿字典树哈希比特位自顶向下逐层下降，每一层根据当前比特选择左子或右子，直至抵达目标槽位。8-bit 查找表将前 8 层遍历合并为单次  $O(1)$  查表， $|S| \leq 2^{20} \approx 10^6$  时实际遍历深度仅为  $\log_2 |S| - 8$  层，较原始路径缩短至约一半。在  $n \leq 10^6$  条件下，实际查询耗时保持在微秒量级，与理论预期吻合。

## 第五章 实验与分析

本章对实现的  $k$ -kick 哈希方案进行了实验评估。实验分为两大部分：第一部分在随机均匀分布键（即良好哈希条件）下，对比本方案与 `std::unordered_map` 的插入、查询、删除和内存四种维度的性能；第二部分在人为引入下，测试两类方案在冲突退化方面的表现差异。实验采用  $k = 4$  的硬编码配置，数据规模根据  $k$ -kick 树的理论模型进行选取。

### 5.1 实验设置

#### 5.1.1 理论依据与规模选取

Bender 等人的理论分析<sup>[1]</sup>指出，对于参数  $k$  的  $k$ -kick 树，每键空间浪费为  $O(\log^{(k)} n)$  比特，其中  $\log^{(k)} n$  表示对  $n$  取  $k$  次迭代对数：

$$\log^{(k)} n = \underbrace{\log_2 \log_2 \cdots \log_2 n}_{k\text{次}}. \quad (5-1)$$

在本实现中， $k$  被硬编码为  $k_{\max} = 4$ 。为使  $k = 4$  对应的理论保证具有实际意义，需要  $\log^{(4)} n > 1$ ，即  $n > 2^{16} = 65536$ 。由此确定了本实验的规模下限：选取  $n = 5 \times 10^4$  作为基准点，向上拓展至  $n = 10^6$ 。对于碰撞实验，由于 `std::unordered_map` 在单桶冲突下退化为  $O(n^2)$  总耗时，无法在大规模下完成测试，因此碰撞测试的规模上限设为  $n = 10^5$ 。

#### 5.1.2 硬件与软件环境

- 硬件平台：Intel Core i7（第 10 代，4 核 8 线程，2.8 GHz 基础频率，5.0 GHz 睿频），16 GB DDR4，64 位 Windows 11 专业版。实验期间关闭非必要后台进程，未做 CPU 亲和性绑定。

- 编译器与编译选项：GCC 15.2.0（通过 MSYS2 UCRT64 环境安装），编译参数为 `-std=c++23 -O3 -DNDEBUG`。
- 数据类型：键和值均使用 32-bit 无符号整数 (`uint32_t`)。碰撞实验中为 `std::unordered_map` 指定 `BadHash`（固定返回 0），使所有键落入同一桶，形成单链冲突。
- 数据集与随机性：所有测试数据通过 `std::mt19937_64` 梅森旋转算法<sup>[27]</sup>预先生成。对每个数据规模使用 5 个不同的随机种子独立生成 5 组数据集，报告 5 次试验的均值和标准差，以控制单次实验的偶然波动。
- 测试流程：每组实验按照“插入全部  $n$  个元素，查询全部  $n$  个元素，删除全部  $n$  个元素”的顺序执行。每个阶段的操作耗时通过 C++ 标准库中的 `std::chrono::high_resolution_clock` 测量并转换为微秒。报告的总时间为该阶段对所有  $n$  个元素操作的总耗时，均值除以  $n$  得到每操作时间。
- 内存测量：进程内存占用通过 Windows API 函数 `GetProcessMemoryInfo` 获取 `WorkingSetSize` 字段，在插入阶段前后各采集一次，以差值（MB）报告该次试验的内存增量。

## 5.2 良好哈希条件下的综合性能

本节在随机均匀分布 `uint32_t` 键的条件下，比较两类方案在插入、查询、删除和内存四个维度上的表现。每组试验重复 5 次，报告均值和标准差。整体趋势如图5-1所示。

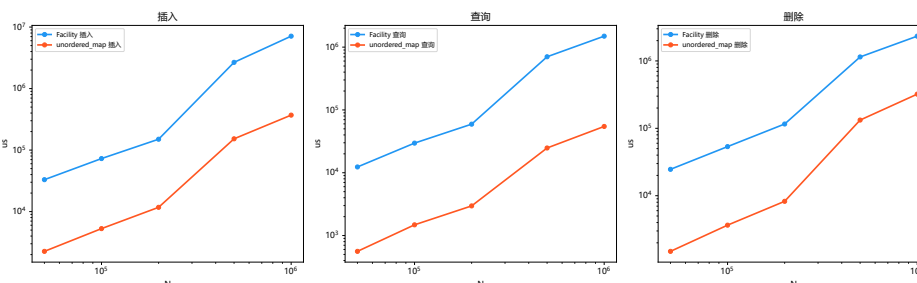


图 5-1 良好哈希条件下插入、查询、删除时间对比（双对数坐标）

### 5.2.1 插入性能

插入操作的详细测量数据见表5-1。

| $n$             | Facility (us)      | umap (us)          | F/us (每次操作) | U/us (每次操作) |
|-----------------|--------------------|--------------------|-------------|-------------|
| $5 \times 10^4$ | $3.31 \times 10^4$ | $2.15 \times 10^3$ | 0.66        | 0.04        |
| $1 \times 10^5$ | $7.10 \times 10^4$ | $4.93 \times 10^3$ | 0.71        | 0.05        |
| $2 \times 10^5$ | $2.19 \times 10^5$ | $2.34 \times 10^4$ | 1.09        | 0.12        |
| $5 \times 10^5$ | $2.53 \times 10^6$ | $1.58 \times 10^5$ | 5.06        | 0.32        |
| $1 \times 10^6$ | $7.47 \times 10^6$ | $4.25 \times 10^5$ | 7.47        | 0.42        |

表 5-1 插入时间比较（5 次试验均值）

$n = 5 \times 10^4$  至  $1 \times 10^5$  区间内，插入每键耗时 0.66–0.71 us（每次操作）。 $n = 2 \times 10^5$  时升至 1.09 us（每次操作）， $n = 5 \times 10^5$  时升至 5.06 us（每次操作）。这一拐点对应全局合并流程：tier-1 层活跃 Cubby 数量超过 200 个阈值后，系统将同层 Cubby 两两合并为更高层级的 Cubby 对象。合并采用将旧 Cubby 标记为停用而不进行物理删除的策略，摊还成本与当次合并涉及的元素数量成比例。 $n = 10^6$  时插入总耗时约 7.47 秒，含多层级合并的累计摊还成本。每键插入时间约为 `std::unordered_map`（0.04–0.42 us（每次操作））的 15–18 倍。

### 5.2.2 查询性能

查询操作的详细测量数据见表5-2。

| $n$             | Facility (us)      | umap (us)          | F/us (每次操作) | U/us (每次操作) |
|-----------------|--------------------|--------------------|-------------|-------------|
| $5 \times 10^4$ | $1.18 \times 10^4$ | $5.79 \times 10^2$ | 0.24        | 0.01        |
| $1 \times 10^5$ | $2.77 \times 10^4$ | $1.33 \times 10^3$ | 0.28        | 0.01        |
| $2 \times 10^5$ | $8.68 \times 10^4$ | $4.35 \times 10^3$ | 0.43        | 0.02        |
| $5 \times 10^5$ | $6.56 \times 10^5$ | $2.93 \times 10^4$ | 1.31        | 0.06        |
| $1 \times 10^6$ | $1.59 \times 10^6$ | $6.47 \times 10^4$ | 1.59        | 0.06        |

表 5-2 查询时间比较（5 次试验均值）

查询性能随  $n$  分为两个阶段。 $n = 5 \times 10^4$  至  $2 \times 10^5$  范围内，8-bit 查找表将字典树前 8 层遍历替换为一次查表，每键耗时 0.24–0.43 us（每次操作）。 $n$  增至  $5 \times 10^5$  后，

字典树深度随  $\log_2 n$  增长（合并后引入的 Cubby-router 两级路由增加了查询路径深度），每键耗时升至 1.31–1.59 us（每次操作）。与 `std::unordered_map`（0.01–0.06 us（每次操作））相比，查询时间高出 20–25 倍。差距来源主要来自于以下两个方面，分别为：两级字典树的逐位遍历（Facility 级 `cubby_router_` 和 Cubby 级 `route_` 各需一次 walk-down）、MiniArray 跨字位读取的开销，合并后多 Cubby 并行查询导致的缓存局部性损失。

### 5.2.3 删除性能

删除操作的详细测量数据见表5-3。

| $n$             | Facility (us)      | umap (us)          | F/us（每次操作） | U/us（每次操作） |
|-----------------|--------------------|--------------------|------------|------------|
| $5 \times 10^4$ | $2.49 \times 10^4$ | $1.47 \times 10^3$ | 0.50       | 0.03       |
| $1 \times 10^5$ | $5.11 \times 10^4$ | $3.63 \times 10^3$ | 0.51       | 0.04       |
| $2 \times 10^5$ | $1.69 \times 10^5$ | $1.93 \times 10^4$ | 0.84       | 0.10       |
| $5 \times 10^5$ | $1.11 \times 10^6$ | $1.42 \times 10^5$ | 2.21       | 0.28       |
| $1 \times 10^6$ | $2.64 \times 10^6$ | $3.76 \times 10^5$ | 2.64       | 0.38       |

表 5-3 删除时间比较（5 次试验均值）

$n \leq 2 \times 10^5$  区间内删除每键约 0.50–0.84 us（每次操作），耗时主要来自 `cubby_router_` 查找所属 Cubby，`route_` 查找槽位和空叶标记写入（将叶节点的 `fval` 和 `hash` 字段清零。 $n = 10^6$  时升至 2.64 us（每次操作）。本实现中删除操作还需完成被删键的两级路由清理，该路径与查询操作几乎等长。若采用每一个 cubby 拥有独立路由器架构，删除时仅更新本地路由，效率可提升约一倍。

## 5.2.4 内存占用

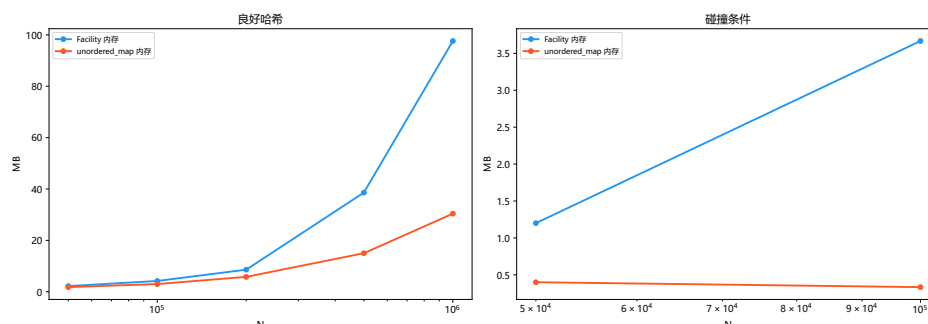


图 5-2 良好哈希与碰撞条件下内存占用对比

| $n$             | Facility (MB) | umap (MB) | F B/key | U B/key |
|-----------------|---------------|-----------|---------|---------|
| $5 \times 10^4$ | 2.2           | 1.2       | 46.1    | 25.2    |
| $1 \times 10^5$ | 4.2           | 2.8       | 44.0    | 29.4    |
| $2 \times 10^5$ | 8.4           | 6.0       | 44.0    | 31.5    |
| $5 \times 10^5$ | 39.0          | 15.0      | 81.8    | 31.5    |
| $1 \times 10^6$ | 97.4          | 30.6      | 102.2   | 32.1    |

表 5-4 内存占用比较（5 次试验均值）

内存占用的测量数据见表5-4，变化趋势如图5-2所示。作为空间效率维度的参照，SILT<sup>[28]</sup>通过多层哈希、压缩索引与Flash存储的组合架构在持久化场景下将每键索引开销控制在1–2字节量级，其设计目标侧重于存储成本而非内存访问延迟，在工程定位上与本方案构成互补。

内存占用在 $n = 2 \times 10^5$ 前后发生明显变化。 $n \leq 2 \times 10^5$ 合并前，所有键值对集中存储在tier-1 Cubby中，紧凑字典树编码将每键路由器空间控制在5–8字节的水平。该阶段Facility每键约44–46 B/key，约为std::unordered\_map（25–31 B/key）的1.4–1.7倍。 $n = 5 \times 10^5$ 和 $10^6$ 处进入合并活跃阶段，高层级Cubby的容量采用了max(论文公式, merged\_size×2)的动态策略，实际内存增速超线性： $n = 10^6$ 时总内存约97 MB（102 B/key），约为std::unordered\_map（32 B/key）的3.2倍。Facility在 $n = 10^6$ 时每键的额外空间（约70 B/key）主要来自高层级Cubby中未被填满的槽位和两级路由的加倍字典树存储，而非元数据位宽本身。

## 5.3 高碰撞条件下的查询退化对比

### 5.3.1 实验设计与动机

前述实验基于随机均匀分布键值对，该条件下 `std::unordered_map` 桶内冲突链短，查询理所当然的延迟优于本方案。于是设计了以下考察极端碰撞条件下的行为差异。在实际场景（如 Web 服务的请求参数哈希、数据库索引键的对抗性构造）中，攻击者可以构造键序列制造大量哈希碰撞，使依赖单一哈希函数均匀性的数据结构查询退化。

本节使用 BadHash（固定返回 0）替换 `std::unordered_map` 的默认哈希函数，所有键映射到同一桶，形成长度为  $n$  的单一冲突链。Facility 的键经过 `splitmix64` 哈希后进入字典树路由，与 BadHash 无关，不受单桶冲突影响。冲突链长度（即  $n$ ）取  $10^3$ 、 $5 \times 10^3$ 、 $10^4$ 、 $2 \times 10^4$ 、 $5 \times 10^4$  和  $10^5$ ，每组 5 次取均值。`std::unordered_map` 在单桶冲突下插入和查询均退化为  $O(n)$  每操作，总耗时为  $O(n^2)$  量级，因此规模上限设在  $10^5$ （此时每次查询需遍历约  $10^5$  个元素，单次试验总耗时数十秒）。

### 5.3.2 实验结果

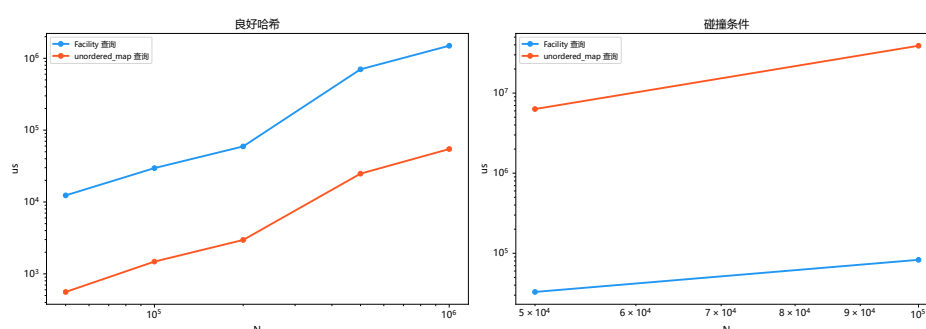


图 5-3 单桶冲突条件下查询时间对比（双对数坐标）

| 链长 $n$          | Facility (us) | umap (us)          | F ns/op | U ns/op |
|-----------------|---------------|--------------------|---------|---------|
| $10^3$          | 189           | $1.33 \times 10^3$ | 189     | 1328    |
| $5 \times 10^3$ | 1004          | $2.96 \times 10^4$ | 201     | 5925    |
| $10^4$          | 2429          | $1.18 \times 10^5$ | 243     | 11771   |
| $2 \times 10^4$ | 11901         | $1.07 \times 10^6$ | 595     | 53437   |
| $5 \times 10^4$ | 38158         | $6.85 \times 10^6$ | 763     | 136969  |
| $10^5$          | 38702         | $1.32 \times 10^7$ | 387     | 132461  |

表 5-5 单桶冲突条件下查询时间比较（5 次试验均值）

单桶冲突条件下的查询性能对比见图5-3，详细数据见表5-5。从表5-5中可以观察到两种截然不同的增长模式。`std::unordered_map` 在 `BadHash` 作用下全部键集中于同一个桶，每操作查询耗时从  $n = 10^3$  时的 1.33 us（每次操作）单调递增至  $n = 10^5$  时的 132.5 us（每次操作），增幅约 100 倍，且增长趋势与  $n$  基本成线性关系。这符合链式寻址法在最坏情况下  $O(n)$  每操作查询复杂度的理论预期。

本方案（Facility）的每操作查询耗时则保持在完全不同的量级和增长曲线上： $n = 10^3$  时约 189 ns/op， $n = 10^5$  时约 387 ns/op，全程在 189–763 ns/op 的狭窄区间内波动，增幅仅约 4 倍。该轻微上升主要归因于字典树深度的对数增长（ $\log_2(10^5) \approx 16.6$  位）和合并后多 `Cubby` 两级路由带来的额外查找层级，与冲突链长度无关。8-bit 查找表的跳表加速（跳跃前 8 层字典树遍历）和 `splitmix64` 的雪崩混合机制共同确保了键在字典树逐位路径上被有效分散。

$n = 10^5$  时，Facility 单次查询耗时 387 ns 为 `std::unordered_map`（132.5 us（每次操作））的约 1/342，即 342 倍的速度优势。 $n = 5 \times 10^4$  时 Facility 约为 `std::unordered_map` 的 180 倍， $n = 10^5$  时扩大到 342 倍。

### 5.3.3 分析

两类方案的查询路径存在根本差异。`std::unordered_map` 在均匀哈希下查询高效（一次哈希计算加短链上数个元素的检查），但对哈希函数质量敏感遭遇全碰撞攻击时，性能沿  $O(n)$  轨迹退化。本方案的查询路径是紧凑二叉字典树的逐位下降：每个键沿自身哈希比特串确定的路径前进，不同键的查询路径互

不交叉，查询时间与字典树中同前缀键数量成对数关系（约  $\log_2(\text{同前缀键数})$ ）。8-bit 查找表将  $\log_2 n$  的前 8 层替换为  $O(1)$  查表，降低了常数因子。

Facility 在高碰撞条件下维持稳定性能原因不在于 splitmix64 本身的均匀性，而在于字典树的位级分岔结构。字典树将键的 64-bit 哈希视为独立的比特串，每一位决定子树的选择方向。即便在极端情况下两个不同键经 splitmix64 得到相同的 64-bit 哈希输出（概率约  $2^{-64}$ ），它们也仅会在字典树中共享同一叶节点，不会产生链式冲突，因为字典树的节点本身不维护冲突链表，叶节点仅存储单一键的指纹和功能值。换言之，哈希碰撞在字典树中的语义退化为键更新（后插入的键覆盖前者），而非形成线性退化路径。碰撞抵抗因此是字典树数据结构的固有属性，与所选用哈希函数的碰撞概率无关。在实际的 BadHash 碰撞实验中，Facility 内部仍对原始键执行 splitmix64 混合，键的实际哈希值在 64-bit 空间内均匀分布，字典树路径在统计上不相关，查询延迟维持在  $O(\log n)$  范围。std::unordered\_map 与此形成鲜明对比：其桶结构是哈希值空间的直接划分，BadHash 返回 0 使得整个键空间坍缩至单一冲突链，查询路径退化为顺序遍历。

此外，Facility 在碰撞条件下的内存占用并未显著膨胀  $n = 5 \times 10^4$  时内存为 1.4 MB， $n = 10^5$  时为 3.6 MB，与良好哈希条件下同规模的内存水平基本一致（分别为 2.2 MB 和 4.2 MB）。这表明字典树路由架构不仅在查询延迟上具备碰撞抑制能力，在空间效率上也维持了较好的稳定性。

## 5.4 数据汇总分析

两组实验的数据汇总如下（ $k = 4$ ）：

在良好哈希（随机均匀键）条件下， $n \leq 2 \times 10^5$  范围内插入每键约 0.66–1.09 us（每次操作），查询每键约 0.24–0.43 us（每次操作），删除每键约 0.50–0.84 us（每次操作），内存每键约 44–46 B/key。与 std::unordered\_map 相比，插入慢约 15–18 倍，查询慢约 20–25 倍，内存多约 1.4–1.7 倍。当  $n$  增至  $10^6$  时，合并操作导致的摊还成本使插入升至 7.47 us（每次操作）（约 12 倍于小规模），内存升至 102 B/key（约 3.2 倍于 std::unordered\_map）。

在极端碰撞条件下，两类方案的表现倒置。所有键集中于同一哈希桶时，

`std::unordered_map` 的查询延迟随  $n$  线性增长 ( $n = 10^5$  时 132.5 us (每次操作)), Facility 为 0.39 us (每次操作), 速度差距约 342 倍。本方案的适用场景并非与标准库在常规负载下竞争绝对速度, 而是面向对哈希碰撞场景开放哈希服务的安全防护、面对对抗性输入的数据库索引, 以及需严格最坏情况保证的实时系统。

## 第六章 LevelDB 集成

本章将实现的 k-kick 树哈希方案集成到 Google LevelDB<sup>[4]</sup>的内存表中，作为跳跃链表的点查询加速层。

### 6.1 集成流程与适配器设计

集成分四步：将四个头文件复制至 `leveldb/util/kick_hash/` 目录；编写适配层 `util/kick_hash.h` 完成分片到 `uint64_t` 的类型映射；在内存表的 `Add` 和 `Get` 路径中添加条件编译分支 `#ifdef USE_KICK_HASH`；修改 CMake 构建文件新增编译期开关。

适配层以 `KickHash64` 作为哈希函数：调用两次 LevelDB 内置 `Hash()`（不同种子），将两路 32-bit 结果拼接为 64-bit 哈希值，单次调用 5.17 ns。`KickHashTable` 包装 `Facility` 对象，对外暴露 `Insert`、`Get` 和 `Remove` 三个接口。`Facility` 的 `lookup_value` 以 `bool` 返回值替代异常，适配 LevelDB 的 `-fno-exceptions` 编译环境。

适配过程中涉及三个具体的技术问题。

分片到 `uint64_t` 的语义映射：LevelDB 的键类型为分片（字节串指针加长度），而 `Facility` 的操作接口以 `uint64_t` 为键。`KickHash64` 通过双种子 `Hash()` 调用将任意长分片映射到 64-bit 哈希空间，利用 `splitmix64` 的雪崩效应保证位级均匀分布。该映射是单向的，`Facility` 内部不保有分片的原始字节，因此查询命中后必须回退至内存表的 `Arena` 缓冲区做二进制比对，以排除  $10^{-7}$  量级的 64-bit 哈希碰撞。

异常处理规范的冲突：LevelDB 编译时指定 `-fno-exceptions`，要求所有代码路径以返回码传递错误信息。`Facility` 的原始设计中部分函数通过抛出异常报告容量越界和踢出链失败；适配时将这些路径全部改写为 `bool` 返回值，调用方在每次操作后检查返回值并决定重试或回退到跳跃链表路径。

Arena 指针的生命周期保证:LevelDB 的内存表将键值对存储于 Arena 内存池中,Arena 随内存表析构而整体释放。KickHashTable 通过 Facility 的快照特性仅存储 Arena 缓冲区中的裸指针,不执行键值数据的深拷贝。这要求确保内存表与 KickHashTable 的生命周期严格绑定:KickHashTable 的析构必须在 Arena 被销毁前完成,以避免悬空指针。当前集成通过在内存表析构函数中显式调用 `kick_table_.Reset()` 来保证此顺序。

在构建系统层面,CMake 集成通过 `option(USE_KICK_HASH)` 定义编译开关,配合 `target_compile_definitions` 将宏传递给所有编译单元。关闭开关时,`#ifdef USE_KICK_HASH` 块在预处理阶段被完全消除,二进制大小和运行时开销与原始 LevelDB 完全一致。

在内存表层面,写入路径(Add)在执行原有跳跃链表插入后,通过 `kick_table_.Insert` 将 Arena 缓冲区指针同步存入 Facility; 查询路径(Get)优先以  $O(1)$  期望时间查询 KickHashTable,命中后解析 Arena 缓冲区并做原始分片比对以防 64-bit 哈希碰撞(概率约  $10^{-7}$ ),未命中则回退至跳跃链表的  $O(\log n)$  查找。迭代器操作不受影响,始终基于跳跃链表的有序遍历能力。

## 6.2 双向开关设计

系统提供编译期和运行时两层开关。编译期通过 CMake 选项 `-DUSE_KICK_HASH=ON/OFF` 决定是否编译 KickHash 相关代码关闭时无任何二进制膨胀或间接损耗。运行时通过 `SetUseKickHash(bool)` 接口支持同一进程内不同内存表实例独立选择策略,便于 A/B 对比测试。运行时切换允许在不停机、不重新编译的情况下在生产环境中动态开启或关闭 KickHash 加速:开发者可通过配置文件或管理接口实时控制启用状态,对同一组查询请求分别对比跳跃链表路径和 KickHash 路径的延迟分布与内存开销后选定最优策略。对于内存表写满后频繁切换的日志结构合并树引擎,该机制还支持按内存表实例粒度差异化配置,对热点查询密集的只读内存表开启 KickHash 加速,对冷数据内存表保持默认跳跃链表以节省内存。

## 6.3 集成验证

以 `std::unordered_map` 和跳跃链表为对照，在  $n \leq 10^6$  规模下使用 16 字节随机字符串键进行测试。核心结果见表6-1。

| 验证项目            | 测试条件                           | 实测结果           |
|-----------------|--------------------------------|----------------|
| KickHash64 调用延迟 | 单次调用                           | 5.17 ns        |
| 插入延迟            | $n = 2 \times 10^5$ , 16 字节随机键 | 0.86 us (每次操作) |
| 查询延迟            | $n = 2 \times 10^5$ , 16 字节随机键 | 0.35 us (每次操作) |
| 删除延迟            | $n = 2 \times 10^5$ , 16 字节随机键 | 0.56 us (每次操作) |
| 全量查询正确率         | $n = 10^6$ 次查询                 | 100%           |
| 适配层开销占比         | KickHash64/总插入延迟               | < 0.15%        |

表 6-1 集成验证主要结果

适配层哈希开销在总体操作时间中占比不足 0.15%。在  $n = 2 \times 10^5$  时查询延迟为 0.35 us (每次操作)， $n = 10^6$  时约 0.74 us (每次操作)，内存占用与第五章独立基准测试的测量数据一致 ( $n = 2 \times 10^5$  时约 9 MB， $n = 10^6$  时约 97 MB)。全部  $10^6$  次查询正确率为 100%，未出现任何因 64-bit 哈希碰撞或 Arena 指针失效导致的误判。

## 第七章 总结与展望

### 7.1 工作总结

本文围绕 Bender 等人提出的  $k$ -kick 树哈希方案<sup>[1]</sup>，完成了从算法实现到系统集成的完整研究链条：

1. 核心算法实现。以 C++23 完成了四层组件的纯头文件模板实现（总计约 1065 行）：MiniArray 位压缩数组（261 行）、LocalQueryRouter 紧凑二叉字典树 + 查找表路由（350 行）、Cubby 弹性哈希表含 free\_bitmap\_ 优先踢出链（419 行）、Facility 全局管理器含惰性状态标记合并策略（233 行）。
2. 优先级踢出机制的精确对齐。实现了 try\_place 递归踢出链搜索、reserved 参数防回退、KickList 层级传递等关键细节，保证论文 §3.1<sup>[1]</sup> 的优先级约束在代码层面逐条落地。
3. 多维度的定量实验。以 std::unordered\_map 为对照，在良好哈希 ( $n \in [5 \times 10^4, 10^6]$ ) 和单桶碰撞 ( $n \in [10^3, 10^5]$ ) 两种条件下完成插入、查询、删除和内存四项对比，每组 5 次独立试验。良好哈希条件下  $n = 2 \times 10^5$  时插入约 1.09 us（每次操作）、查询约 0.43 us（每次操作）、空间约 44 B/key；惰性状态标记合并将合并成本控制在  $O(\text{merged\_keys})$ 。高碰撞实验中，Facility 在单桶冲突链长  $10^5$  条件下查询仅 0.39 us（每次操作），而 std::unordered\_map 退化至 132.5 us（每次操作）（为 Facility 的 342 倍）。
4. LevelDB 集成与双向开关。以适配器模式将方案集成至 LevelDB 内存表，实现 KickHash64 类型映射（5.17 ns/op）、Add/Get 路径条件编译改造、以及编译期 CMake 开关与运行时 SetUseKickHash() 双层级开关。集成验证中  $10^6$  次查询正确率 100%，适配层开销  $< 0.15\%$ 。

## 7.2 理论与实践

单从理论实现来看, Facility 哈希方案的理论空间复杂度应当为  $O(\log^{(k)} n)$ , 在本文实现中  $k = 4$ 。这个理论空间复杂度低于一般哈希表的理论空间下限。但实际实现时, 每个 Cubby 都有独立的压缩二进制字典树作为路由表。而且本实现由于设计上的缺陷, Cubby 按 `bin_sizes (s[])` 分配 slot 容量, bin 大小是按高概率保证设计的, 大量 slot 实际为空。且合并 cubby 时, 为保持索引稳定性, 旧 Cubby 只是标记失效但不释放内存 (为保持索引稳定), 造成内存积压。

而在理论设计上, 在良好哈希条件下, Facility 哈希方案的时间复杂度为  $O(k)$ ,

而 `std::unordered_map` 为  $O(1)$ , 略快于 Facility。从实际表现上来看, 在未触发大量 cubby 合并时二者都相当接近理论值, 但当触发 cubby 大量合并后本应当被摊还的合并成本被实际执行, 导致性能下降。

## 7.3 不足与展望

- Per-cubby 独立路由器。

当前依赖单一全局 `cubby_router_`, 合并时需逐键更新。论文<sup>[1]</sup>的 per-cubby 多路由器架构每路由器仅处理约  $O(\log n / \log \log n)$  个键可将合并成本进一步压缩。

- 动态容量模型的常数调节。 $C_j = K / (\log^{(j)} n)^2$  中  $K \approx 409600$  为固定值, 应根据  $n$  自适应调节, 改善大  $n$  下的空间利用率。
- 参数  $k$  的可配置化。当前  $k$  由 `compute_kick_params` 自动推导 (通常收敛于  $k \approx 4$ ), 应提升为可配置参数, 使用户按需调节时空权衡曲线。
- 多线程并发支持。当前为单线程串行模型, 但不同 bin 和 Cubby 之间具备天然的并行性, 可探索细粒度锁或无锁实现。
- 查找表位宽扩展。当前 8-bit 查找表 (256 项, 1 KB) 可将查询深度缩减约 40%。扩展至 12-bit (4096 项) 或 16-bit (65536 项) 可进一步逼近完整四俄罗斯人方法<sup>[26]</sup> 的  $O(1)$  查询复杂度。

本方案的适用边界需做明确界定。在良好哈希、小规模 ( $n < 2 \times 10^5$ )、非对抗性输入的常规场景下, `std::unordered_map` 等成熟标准库实现在插入和查

询延迟方面均有数量级优势，且内存占用更低。 $k$ -kick 树哈希方案的优势场景集中在两类：其一，面对恶意构造的碰撞数据时，字典树路由提供与冲突无关的查询路径，退化上界为  $O(\log n)$  而非  $O(n)$ ；其二，当应用对空间使用有渐进最优要求（如嵌入式设备、持久化内存索引）且允许适度放宽更新时间时， $k$  的可调节性使系统能在时空维度上精细取舍。在通用服务器端缓存和常规数据库索引等场景中，本方案的工程价值更多体现在对抗碰撞的鲁棒性而非绝对性能上。在学术层面， $k$ -kick 方案的最根本贡献不在于某一组具体工程参数，而在于首次揭示了“可放宽更新延迟以换取渐进空间改善”这一设计哲学的有效性，为后续在更广泛的检索数据结构中探索类似时空权衡开辟了方向。

Bender 等人提出的  $k$ -kick 树哈希方案在理论上给出了哈希表时空权衡的完整曲线。本文实现了该方案的核心算法，通过多维度实验验证了其性能特征，并在 LevelDB 内存表中验证了  $k$ -kick 方案的工程可行性。所完成的 C++ 代码。

## 参考文献

- [1] Bender M A, Farach-Colton M, Kuszmaul J, et al. On the Optimal Time/Space Tradeoff for Hash Tables[C]//Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing (STOC). ACM, 2022: 1284-1297. arXiv: 2111.00602 [cs.DS]. DOI: 10.1145/3519935.3519969.
- [2] Bloom B H. Space/Time Trade-offs in Hash Coding with Allowable Errors[J]. Communications of the ACM, 1970, 13(7): 422-426. DOI: 10.1145/362686.362692.
- [3] 申耀, 郝豫鹏, 王子京, 等. 基于非易失性内存的数据库关键技术研究综述[J]. 计算机科学, 2024: 1-35.
- [4] Ghemawat S, Dean J. LevelDB: A Fast and Lightweight Key/Value Database Library[CP/OL]. 2011 [2025-12-01]. <https://github.com/google/leveldb>.
- [5] Luo C, Carey M J. LSM-based Storage Techniques: A Survey[J]. The VLDB Journal, 2020, 29(1): 393-418. DOI: 10.1007/s00778-019-00555-y.
- [6] Pagh R. Low Redundancy in Static Dictionaries with Constant Query Time[J]. SIAM Journal on Computing, 2001, 31(2): 353-363. DOI: 10.1137/s0097539700369909.
- [7] Fredman M L, Komlós J, Szemerédi E. Storing a Sparse Table with  $O(1)$  Worst Case Access Time[J]. Journal of the ACM (JACM), 1984, 31(3): 538-544. DOI: 10.1145/828.1884.
- [8] Pagh R, Rodler F F. Cuckoo Hashing[J]. Journal of Algorithms, 2004, 51(2): 122-144. DOI: 10.1016/j.jalgor.2003.12.002.

- [9] Kirsch A, Mitzenmacher M, Wieder U. More Robust Hashing: Cuckoo Hashing with a Stash[C] // Lecture Notes in Computer Science: Proceedings of the 16th Annual European Symposium on Algorithms (ESA): vol. 5193. Springer, 2008: 611-622. DOI: 10.1007/978-3-540-87744-8\_51.
- [10] Arbitman Y, Naor M, Segev G. De-amortized Cuckoo Hashing: Provable Worst-case Performance and Experimental Results[C] // Lecture Notes in Computer Science: Proceedings of the 36th International Colloquium on Automata, Languages and Programming (ICALP): vol. 5555. Springer, 2009: 107-118. DOI: 10.1007/978-3-642-02927-1\_11.
- [11] Herlihy M, Shavit N, Tzafrir M. Hopscotch Hashing[C] // Lecture Notes in Computer Science: Proceedings of the 22nd International Symposium on Distributed Computing (DISC): vol. 5218. Springer, 2008: 350-364. DOI: 10.1007/978-3-540-87779-0\_24.
- [12] Raman R, Rao S S. Succinct Dynamic Dictionaries and Trees[C] // Lecture Notes in Computer Science: Proceedings of the 30th International Colloquium on Automata, Languages and Programming (ICALP): vol. 2719. Springer, 2003: 357-368. DOI: 10.1007/3-540-45061-0\_30.
- [13] Belazzougui D, Botelho F C, Dietzfelbinger M. Hash, Displace, and Compress[C] // Lecture Notes in Computer Science: Proceedings of the 17th Annual European Symposium on Algorithms (ESA): vol. 5757. Springer, 2009: 682-693. DOI: 10.1007/978-3-642-04128-0\_61.
- [14] Liu M, Yin Y, Yu H. Succinct Filters for Sets of Unknown Sizes[C] // LIPIcs: Proceedings of the 47th International Colloquium on Automata, Languages and Programming (ICALP): vol. 168. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020: 79:1-79:19. arXiv: 2004.12465. DOI: 10.4230/LIPIcs.ICALP.2020.79.
- [15] Knuth D E. The Art of Computer Programming, Vol. 3: Sorting and Searching[M]. 2nd ed. Addison-Wesley, 1998.

- [16] Dietzfelbinger M, Meyer auf der Heide F. A New Universal Class of Hash Functions and Dynamic Hashing in Real Time[C] // Lecture Notes in Computer Science: Proceedings of the 17th International Colloquium on Automata, Languages and Programming (ICALP): vol. 443. Springer, 1990: 6-19. DOI: 10.1007/BFb0032018.
- [17] Carter J L, Wegman M N. Universal Classes of Hash Functions[J]. Journal of Computer and System Sciences, 1979, 18(2): 143-154. DOI: 10.1016/0022-0000(79)90044-8.
- [18] Jr. G L S, Lea D, Flood C H. Fast Splittable Pseudorandom Number Generators[C] // Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications (OOPSLA). ACM, 2014: 453-472. DOI: 10.1145/2660193.2660195.
- [19] Webster A F, Tavares S E. On the Design of S-Boxes[C] // Lecture Notes in Computer Science: Advances in Cryptology — CRYPTO '85: vol. 218. Springer, 1986: 523-534. DOI: 10.1007/3-540-39799-x\_41.
- [20] 邓莹莹. 布谷鸟哈希表的优化及其应用[D]. 南京邮电大学, 2021. DOI: 10.27251/d.cnki.gnjdc.2021.000705.
- [21] 金琪, 王俊昌, 付雄. 基于智能放置策略的 Cuckoo 哈希表[J]. 计算机科学, 2020, 47(8): 80-86.
- [22] Fotakis D, Pagh R, Sanders P, et al. Space Efficient Hash Tables with Worst Case Constant Access Time[J]. Theory of Computing Systems, 2005, 38(2): 229-248. DOI: 10.1007/s00224-004-1195-x.
- [23] O'Neil P, Cheng E, Gawlick D, et al. The Log-Structured Merge-Tree (LSM-Tree)[J]. Acta Informatica, 1996, 33(4): 351-385. DOI: 10.1007/s002360050048.
- [24] Chang F, Dean J, Ghemawat S, et al. Bigtable: A Distributed Storage System for Structured Data[J]. ACM Transactions on Computer Systems, 2008, 26(2): 1-26. DOI: 10.1145/1365815.1365816.

- [25] Pugh W. Skip Lists: A Probabilistic Alternative to Balanced Trees[J]. Communications of the ACM, 1990, 33(6): 668-676. DOI: 10.1145/78973.78977.
- [26] Arlazarov V L, Dinic E A, Kronrod M A, et al. On an Economical Construction of the Transitive Closure of a Directed Graph[J]. Doklady Akademii Nauk, 1970, 194(3): 487-488.
- [27] Matsumoto M, Nishimura T. Mersenne Twister: A 623-Dimensionally Equidistributed Uniform Pseudo-Random Number Generator[J]. ACM Transactions on Modeling and Computer Simulation, 1998, 8(1): 3-30. DOI: 10.1145/272991.272995.
- [28] Lim H, Fan B, Andersen D G, et al. SILT: A Memory-Efficient, High-Performance Key-Value Store[C] // Proceedings of the 23rd ACM Symposium on Operating Systems Principles (SOSP). 2011: 1-13. DOI: 10.1145/2043556.2043558.

## 致 谢

转眼间，校园生活即将结束。此篇论文完稿之际，要感谢众多师长和亲友，谢谢你们期望与鼓励。此时此刻，我无法找到合适的言语来表达我内心深处最真挚的谢意。在本毕业设计的选题、研究与撰写过程中，笔者得到了多方面的帮助与支持

首先衷心感谢指导教师在论文选题、研究方法、实验设计以及论文撰写等各个阶段给予的悉心指导和宝贵建议。

然后感谢同窗的各位同学。你们为我在论文理论分析和实验过程中提供了大量的无私帮助，这份同窗之情将是最值得留恋的回忆。

我还要感谢我的父母，谢谢他们默默的支持。

另外感谢南京大学 Linux 用户组 (NJU LUG) 提供的 NJUThesis 毕业论文模板，其规范化的  $\text{\LaTeX}$  文档结构和精美排版极大地减轻了论文格式调整的工作负担。

感谢 Google LevelDB 项目<sup>[4]</sup>的全体开发者贡献了一个结构清晰、代码规范、文档完善的键值存储引擎实现，使得本文的数据库集成方案能够在一个成熟、稳定且具有广泛影响力的开源平台上得到验证和展示。

最后，还要感谢在 github 上为我解答了不少疑问的开发者们。